JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class TimerTask

java.lang.Object
    java.util.TimerTask

All Implemented Interfaces:

Runnable

```
public abstract class TimerTask
extends Object
implements Runnable
```

A task that can be scheduled for one-time or repeated execution by a Timer.

Since:

1.3

See Also:

Timer

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	TimerTask()	Creates a new timer task.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
boolean	**cancel**()	Cancels this timer task.
abstract void	**run**()	The action to be performed by this timer task.
long	**scheduledExecutionTime**()	Returns the *scheduled* execution time of the most recent *actual* execution of this task.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

TimerTask

```
protected TimerTask()
```

Creates a new timer task.

Method Detail

run

```
public abstract void run()
```

The action to be performed by this timer task.

Specified by:

`run` in interface `Runnable`

See Also:

`Thread.run()`

cancel

```
public boolean cancel()
```

Cancels this timer task. If the task has been scheduled for one-time execution and has not yet run, or has not yet been scheduled, it will never run. If the task has been scheduled for repeated execution, it will never run again. (If the task is running when this call occurs, the task will run to completion, but will never run again.)

Note that calling this method from within the `run` method of a repeating timer task absolutely guarantees that the timer task will not run again.

This method may be called repeatedly; the second and subsequent calls have no effect.

Returns:

`true if this task is scheduled for one-time execution and has not yet run, or this task is scheduled for repeated execution. Returns false if the task was scheduled for one-time execution and has already run, or if the task was never scheduled, or if the task was already cancelled. (Loosely speaking, this method returns true if it prevents one or more scheduled executions from taking place.)`

scheduledExecutionTime

```
public long scheduledExecutionTime()
```

Returns the *scheduled* execution time of the most recent *actual* execution of this task. (If this method is invoked while task execution is in progress, the return value is the scheduled execution time of the ongoing task execution.)

This method is typically invoked from within a task's run method, to determine whether the current execution of the task is sufficiently timely to warrant performing the scheduled activity:

```
public void run() {
    if (System.currentTimeMillis() - scheduledExecutionTime() >=
        MAX_TARDINESS)
            return;  // Too late; skip this execution.
    // Perform the task
}
```

This method is typically *not* used in conjunction with *fixed-delay execution* repeating tasks, as their scheduled execution times are allowed to drift over time, and so are not terribly significant.

Returns:

the time at which the most recent execution of this task was scheduled to occur, in the format returned by Date.getTime(). The return value is undefined if the task has yet to commence its first execution.

See Also:

`Date.getTime()`

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.